QR Code & Deep Link Hijacking in Android

Architecture, Flow & Implementation Plan

Security of Mobile Devices, Assignment 3

Matheas-Roland Borsos • Vlad-Constantin Comârlău | SAS1

1. Introduction

The project has two parts. An **offensive proof-of-concept** shows that a malicious QR code can hijack a poorly configured custom-scheme deep link in an Android app bybypassing local authentication or stealing an OAuth callback token. A **defensive auditor** parses `AndroidManifest.xml` files and flags the insecure patterns that make the attack possible (exported components without permissions, custom-scheme deep links, missing `autoVerify`). The two halves share a deliberately vulnerable app.

2. System Architecture

The system splits into three groups: the **attacker side** produces the malicious QR code and a colliding companion app; the **Android device** hosts the vulnerable target app, where the OS Intent Resolver decides which handler receives the incoming deep link; the **auditor** runs on the developer host, pulls the target's manifest over ADB and reports the insecure configurations.

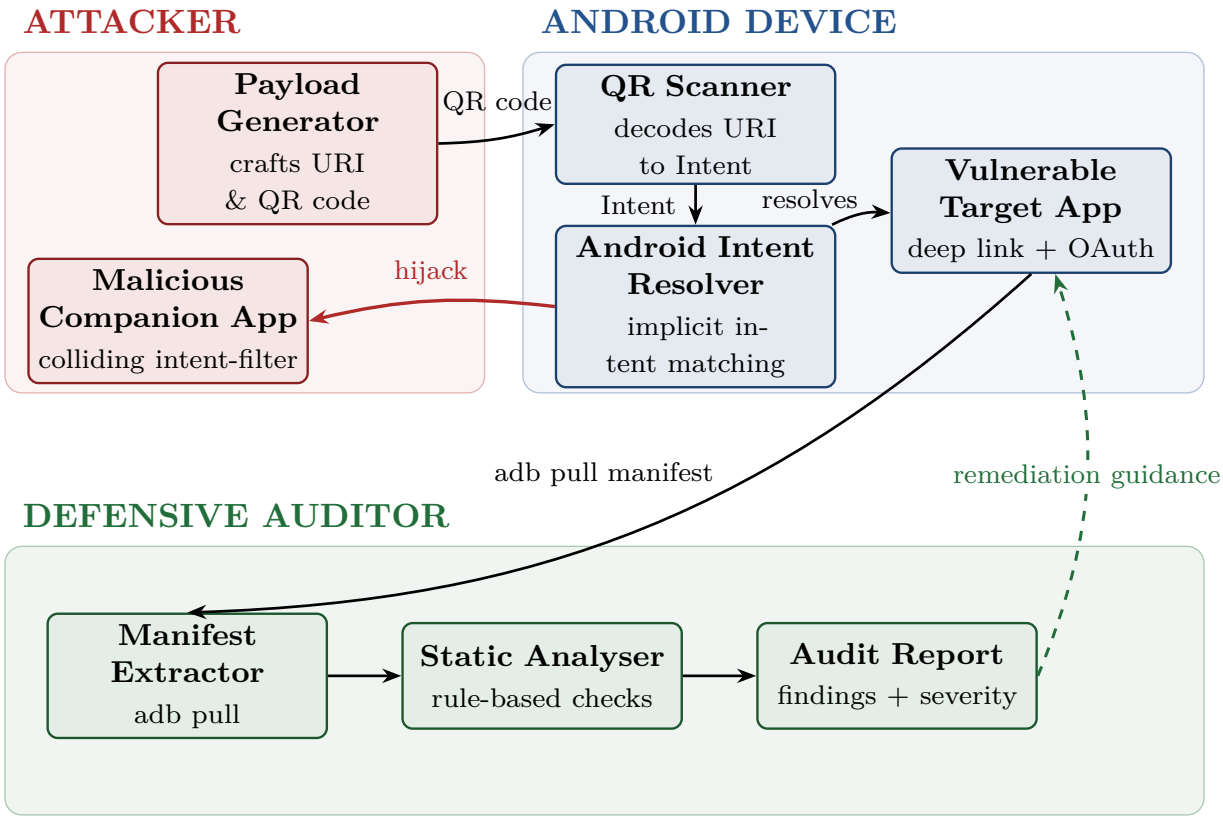

Figure 1. Component architecture

3. Data & Control Flow

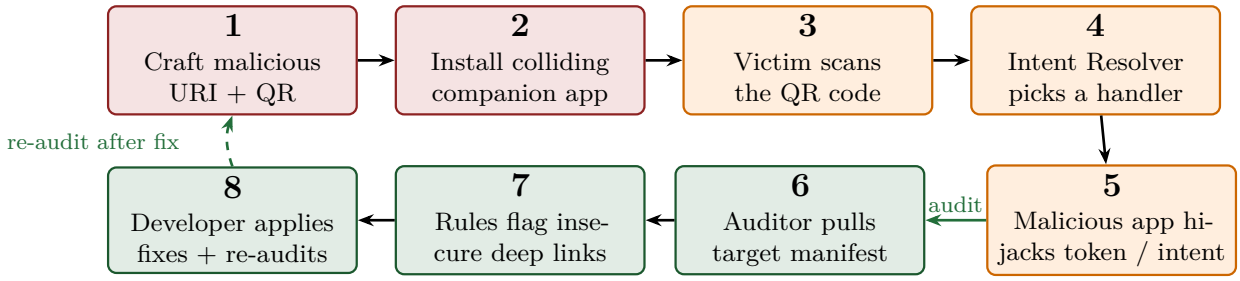

Figure 2. Numbered end-to-end flow through the components of Figure 1.

STEPS:

1. **Craft payload:**

   the Payload Generator builds a deep link (e.g. `smdpoc://oauth/callback?...`) and encodes it as a QR code.

2. **Install colliding handler:**

   a Malicious Companion App is installed on the victim device with an `intent-filter` for the same scheme.

3. **Victim scans:**

   the user scans the QR code; the scanner converts it into an implicit `Intent` (`ACTION_VIEW`).

4. **Intent resolution:**

   the Android Intent Resolver finds two matching handlers; with no origin verification, the malicious one can win or appear in the chooser.

5. **Hijack:**

   the malicious app reads the embedded OAuth token, or forges an intent to a protected component, bypassing the local-auth screen.

6. **Manifest extraction:**

   separately, the auditor pulls `AndroidManifest.xml` from the target app over `adb`.

7. **Static analysis:**

   rules flag insecure patterns: exported components without permissions, custom-scheme deep links, missing `android:autoVerify`.

8. **Remediate & re-audit:**

   developers apply mitigations (verified App Links, explicit intents, permissions) and re-run the audit to confirm findings clear.

4. Implementation Plan

The work is organised into four short phases. One of us made the offensive PoC, and the other the defensive auditor; both of us worked on the vulnerable app.

Phase 0 — Setup

1. Create a Git repository.
2. Lay out the repo: `/target-app`, `/attacker-app`, `/payload-gen`, `/auditor`, `/docs`; scaffold the Android Studio and Python projects.

Phase 1 — Vulnerable Target App

3. Build a small Android app with an insecure deep-link handler on a custom URI scheme (`smdpoc://`) and a mock OAuth callback delivered through an implicit intent.
4. Add one `exported` "internal" activity with no permission guard, reachable without completing the local-auth screen.
5. Document the planted vulnerabilities in `/docs`.

Phase 2 — Offensive Proof-of-Concept

6. Implement a small Payload Generator (Python script) that builds the malicious URI and renders it as a QR code with the `qrcode` library.
7. Build the Malicious Companion App: a single activity declaring an `intent-filter` that collides with the target's scheme and logs the received intent / token.
8. Record one short demo run for each attack (token interception, intent injection).

Phase 3 — Defensive Auditor

9. Write a Python script that pulls `AndroidManifest.xml` from installed apps via `adb` + `apkanalyzer` and parses it.
10. Implement a small set of rules: exported components without permissions, custom-scheme deep links, missing `android:autoVerify`, `BROWSABLE` on sensitive activities.
11. Emit a human-readable report (and JSON) with severity per finding.

Phase 4 — Validation & Delivery

12. Run the auditor against the vulnerable app and confirm every planted issue is flagged; it time allows run it against one or two benign apps as a sanity check.
13. Write the final report and slides.